Il Linguaggio C#

TUTORIAL PER PROGRAMMATORI JAVA

ANGELO CROATTI

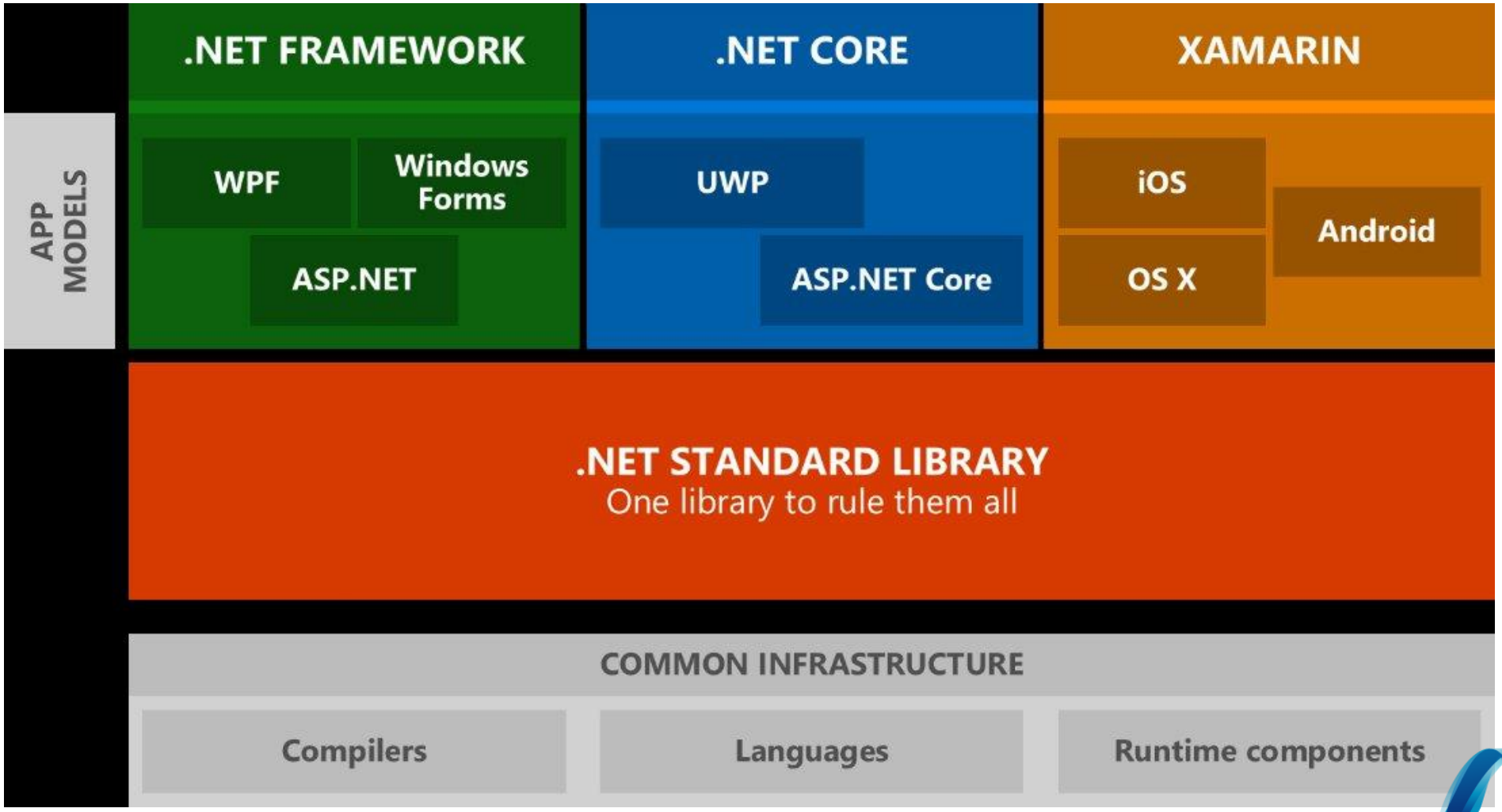
a.croatti@unibo.it

ALMA MATER STUDIORUM – UNIVERSITÀ DI BOLOGNA
Corso di studi in Ingegneria e Scienze Informatiche

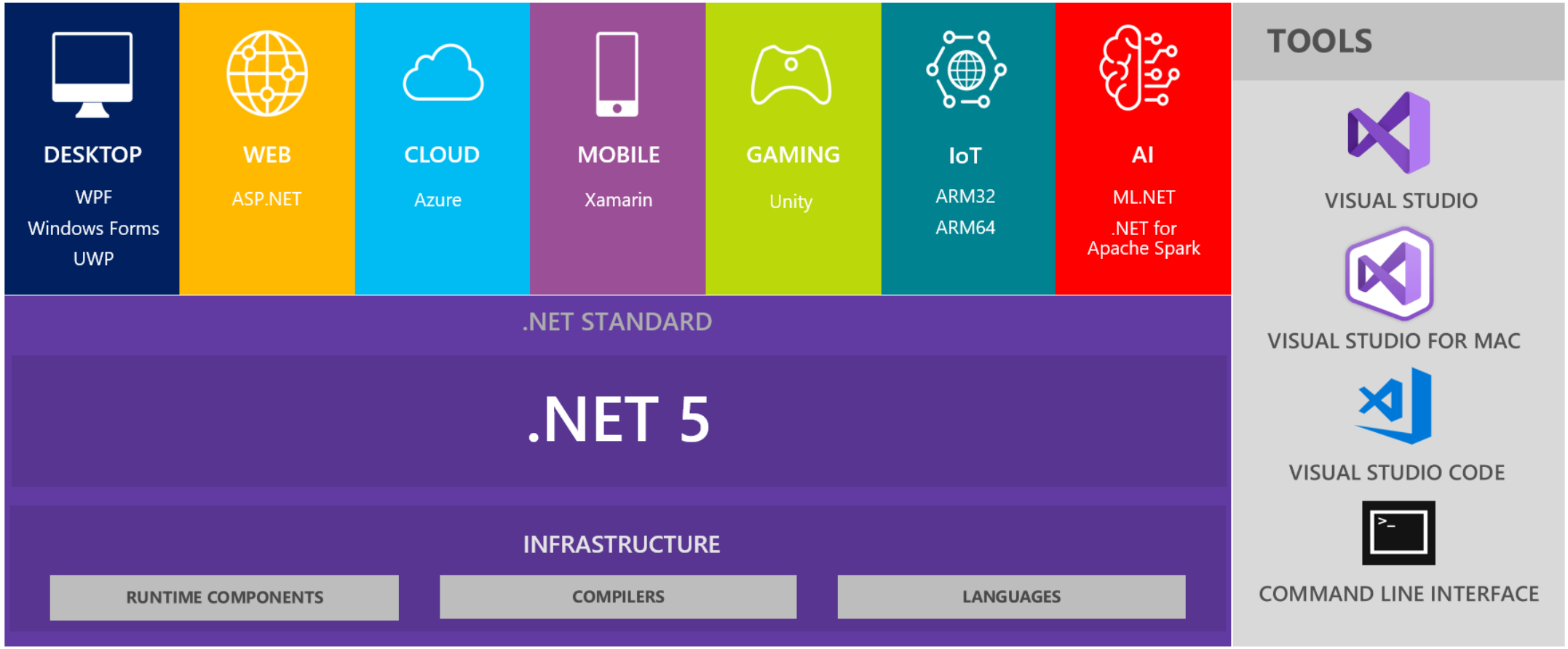
Programmazione ad Oggetti – A.A. 2019/2020

OVERVIEW

Microsoft .NET Development Platform

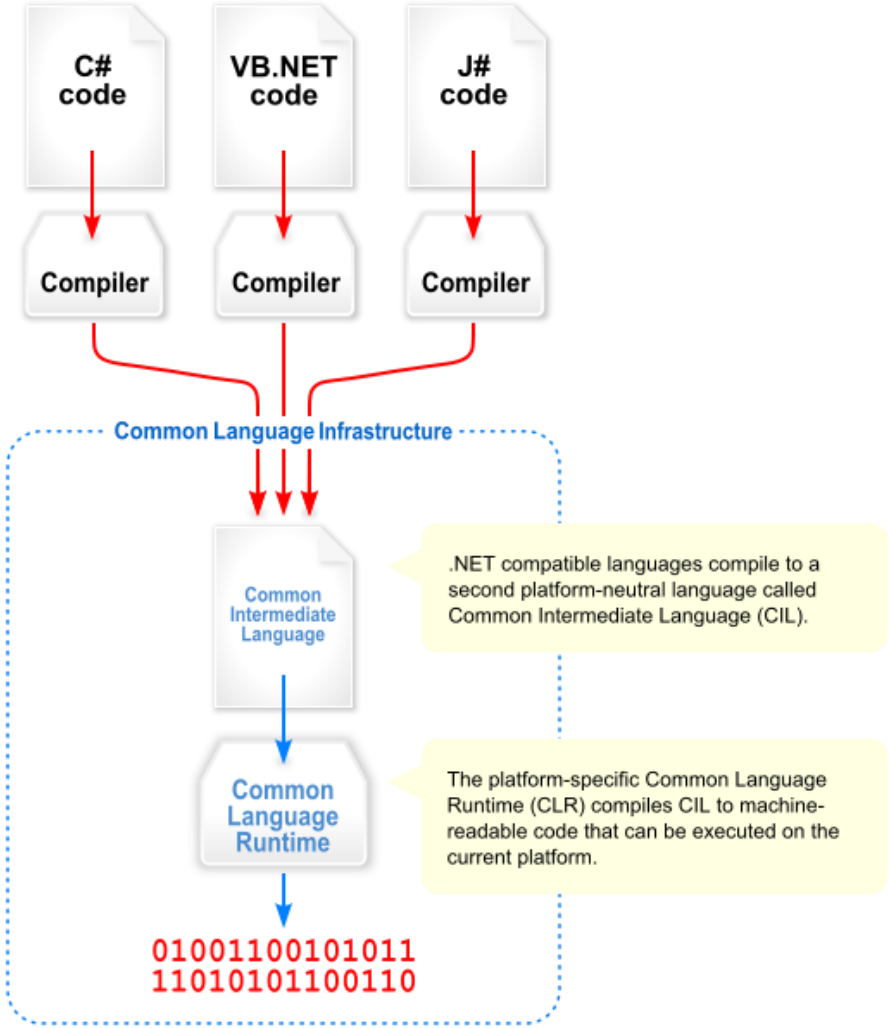


Microsoft .NET Development Platform



Microsoft .NET

- Software framework sviluppato da Microsoft
  - Include un'ampia Class Library e consente di compilare software scritti in diversi linguaggi supportati (principalmente C#).
- I programmi scritti per .NET sono eseguiti in un ambiente virtuale noto come CLR (Common Language Runtime)
  - Fornisce servizi per la gestione della memoria, la gestione delle eccezioni…
  - I programmi sono compilati in un linguaggio intermedio chiamato CIL (Common Intermediate Language)
  - Il codice CIL è poi compilato Just-in-Time (JIT) in codice macchina dalla CLR in fase di esecuzione, con riferimento alla specifica piattaforma, iniettando ciò che occorre del Framework Class Library (FCL).
- Analogia con ecosistema Java
  - Java ↔ C#
  - JDK ↔ Class Library
  - Bytecode ↔ CIL
  - JVM ↔ CLR



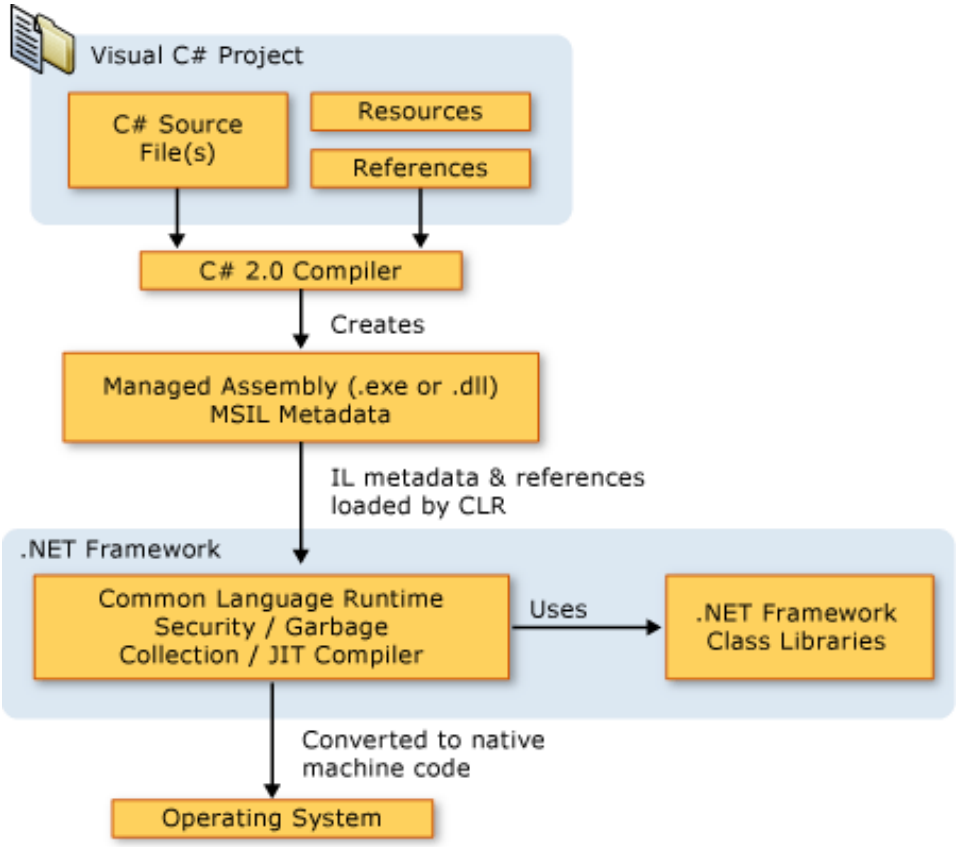
.NET Framework, Standard & Core

- **.NET Framework** è l'ambiente di esecuzione della piattaforma .NET integrato in Windows
  - L'ultima versione è la 4.8, anche se la versione più comunemente usata è la 4.7.2
  - Essendo profondamente integrato in Windows, non è multipiattaforma, né open-source
    - Il **progetto Mono** è un'implementazione open-source e cross-platform del .NET Framework
    - È attualmente supportata da Xamarin, compagnia controllata da Microsoft
  - Nonostante .NET Core e .NET 5 lo rendano «datato», non è deprecato, in quanto molti prodotti Microsoft tutt'ora lo utilizzano
- **.NET Core** è una implementazione completamente riscritta, open-source e multipiattaforma della piattaforma .NET
  - È un prodotto realizzato ufficialmente da Microsoft e supportato dalla .NET Foundation
  - L'implementazione CoreCLR su cui si basa risulta notevolmente più performante della CLR originale
  - L'ultima versione che mantiene il nome «Core» è la 3.1
    - Da **.NET 5** in poi si chiama semplicemente «.NET», nel tentativo di unificare la codebase
- **.NET Standard** è una specifica formale delle API messe a disposizione dalle diverse piattaforme .NET (.NET Framework, .NET Core, Mono, Xamarin, UWP, Unity)
  - Poiché modella le API comuni, offre un subset delle funzionalità esposte da ciascun target
  - L'ultima versione a supportare .NET Framework è la 2.0, mentre la più recente è la 2.1
  - Con .NET 5 si intende abbandonare la specifica .NET Standard in favore semplicemente di .NET

Linguaggio C# - Caratteristiche generali

- Linguaggio (general-purpose) principale di .NET
  - Prevalentemente basato sul paradigma ad oggetti, ma dalle ultime versioni è definibile come linguaggio multi-paradigma
  - È standardizzato Ecma (ECMA-334) e USI (ISO/IEC 23270:2006)
  - Nato nel 2000, è attualmente disponibile nella versione 7.0 (datata 2017)
  - Attualmente è disponibile per tutte le piattaforme (Windows, Linux, …) e il relativo compilatore è stato rilasciato come open-source code.
  - L'IDE principale per lo sviluppo in C# è Microsoft Visual Studio

- Dal Sorgente all'Esecuzione
  - Il compilatore C# produce codice intermedio (CIL)
  - Codice e risorse (es. immagini) formano uno o più Assembly
    - contenuti in appositi file (.exe o .dll in Windows)
  - Il programma C# può utilizzare la Class Library che è messa a disposizione tramite il framework di .NET
  - La CLR carica il codice CIL e ne traduce le parti da eseguire in linguaggio macchina, compilandole mediante JIT



Un esempio: C# > CIL > ASM

Sorgente (C#)

```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hello World!");
    }
}
```

```
.method private hidebysig static void  Main(string[] args) cil managed
{
  .entrypoint
  // Code size       13 (0xd)
  .maxstack  8
  IL_0000:  nop
  IL_0001:  ldstr      "Hello World!"
  IL_0006:  call       void [mscorlib]System.Console::WriteLine(string)
  IL_000b:  nop
  IL_000c:  ret
} // end of method Program::Main
```

Intermediate Language (IL)

```
00000000 55                push        ebp
00000001 8B EC             mov         ebp,esp
00000003 57                push        edi
00000004 56                push        esi
00000005 53                push        ebx
00000006 83 EC 30          sub         esp,30h
00000009 33 C0             xor         eax,eax
0000000b 89 45 F0          mov         dword ptr [ebp-10h],eax
0000000e 33 C0             xor         eax,eax
00000010 89 45 E4          mov         dword ptr [ebp-1Ch],eax
00000013 89 4D C4          mov         dword ptr [ebp-3Ch],ecx
00000016 83 3D 80 88 A1 00 00 cmp          dword ptr ds:[00A18880h],0
0000001d 74 05             je          00000024
0000001f E8 22 11 34 79    call        79341146
00000024 90                nop
00000025 8B 0D 7C 30 37 02 mov          ecx,dword ptr ds:[0237307Ch]
0000002b E8 FC 7E 66 78    call        78667F2C
00000030 90                nop
00000031 90                nop
00000032 8D 65 F4          lea         esp,[ebp-0Ch]
00000035 5B                pop         ebx
00000036 5E                pop         esi
00000037 5F                pop         edi
00000038 5D                pop         ebp
00000039 C3                ret
```

Codice Macchina e ASM (Intel 80386)

Quale Ambiente di Sviluppo (IDE) per .NET ?

- In ambienti Windows o MacOS, l'IDE usato più comunemente per .NET è **Visual Studio**
  - Molto maturo, attualmente alla versione 16.8 (2019)
  - Closed source, versione Community gratuita + versioni a Professional ed Enterprise a pagamento
    - Gli studenti UNIBO possono ottenere una licenza Enterprise gratuitamente tramite Microsoft Azure Dev Tools for Teaching
  - Supporta molti linguaggi: C, C#, F#, Vb.NET, Python, etc..
  - Designer WYSIWYG per WinForms & WPF molto flessibile ed efficace
  - Ottimo per sviluppare... in/per ambienti Windows

- L'IDE **Rider**, di JetBrains, è forse l'alternativa migliore a Visual Studio
  - Funziona sia su Windows, che su MacOS, che su Linux
  - È disponibile **solo** previo acquisto di una licenza
    - È però possibile registrarsi a **JetBrains** con un account **@(studio.)unibo.it** e ottenere una licenza gratuita https://www.jetbrains.com/community/education/#students
  - Molto comodo per chi è abituato a IntelliJ Idea e/o Android Studio

IDE alternativi per .NET

- È possibile configurare opportunamente editor come:
  - Visual Studio **Code**
  - **Atom**
  - **Sublime Text**

  mediante l'installazione di appositi plugin (e.g. OmniSharp) dai rispettivi Plugin-Store/Marketplace

C# e Java

- Entrambi sono linguaggi object-oriented, class based
- Progettati per essere eseguiti su macchina virtuale (CLR e JVM)
- Entrambi supportano la Garbage Collection e i meccanismi di Exception Handling
- Appartengono alla categoria dei linguaggi curly brace (come C, C++, …)
- Sono entrambi linguaggi **statically** and **strongly** typed
- Entrambi **non** supportano l'ereditarietà multipla
- …

PARTE 1

Analogie e differenze tra C# e Java

Convenzioni di scrittura del codice

- Una sola istruzione/dichiarazione per linea
  - Indentazione con spazi al posto della tabulazione
  - Separazione di ciascuna definizione di metodo dalle altre con almeno una linea vuota
- Parentesi graffa aperta
  - (Java) in fondo alla linea precedente
  - (C#) a capo su nuova linea
- Variabili locali e parametri dei metodi
  - camelCase
- Tipi (es. classi/interfacce)
  - PascalCase
- Metodi
  - (Java) camelCase
  - (C#) PascalCase

» https://msdn.microsoft.com/en-us/library/ff926074.aspx

```
using System;

namespace it.unibo.oop.example01
{
    class Program
    {
        private static int mySum;

        public static void Main(string[] args)
        {
            mySum = SumTwoNum(10, 5);

            Console.WriteLine(mySum.ToString());
        }

        private static int SumTwoNum(int firstPar, int secondPar)
        {
            return firstPar + secondPar;
        }
    }
}
```

Struttura dei programmi

ANALOGIE

- Non vi sono file «header» (come accade invece in C/C++)
  - il codice tutto è scritto «in-line»
- Classi raggruppate all'interno di un contenitore
  - «**namespace**» in C#, «**package**» in Java
- Metodo «main» come entry-point
  - In C# inizia con la 'M' maiuscola, come è convenzione in .Net per i metodi pubblici
  - Deve essere definito con il modificatore «**static**» e può accettare i command-line parameters

```
public static void Main()
{
   //TODO
}

public static void Main(string[] args)
{
   //TODO
   string par1 = args[0];
}
```

DIFFERENZE

- In C# la struttura *fisica* di file/directory non deve corrispondere a quella *logica* di classi/namespace
  - Non vi sono restrizioni al numero di classi e namespace (annidati o meno) che possono essere contenuti in un file
  - Per importare classi contenute in namespace non presenti in quello corrente, si utilizza la keyword «**using**» (equivalente a «import» in Java)
- In C# è possibile dividere una classe fra più file
  - Più file possono specificare lo stesso nome di classe preceduto dalla keyword «**partial**»
  - In caso di ereditarietà, è sufficiente che solo una delle parti definisca la classe base

Struttura dei programmi – Esempi

```java
package it.unibo.oop.example01;

public class Program {
     /*
      * Main method
      * Application Entry-Point
      */
     public static void main(String[] args) {
         //Print a message on the StdOut
         System.out.println("Hello, World!");
     }
}
```

```csharp
using System;

namespace it.unibo.oop.example01
{
    class Program
    {
        /*
         * Main method
         * Application Entry-Point
         */
        static void Main(string[] args)
        {
            //Print a message on the StdOut
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Console I/O

ANALOGIE

- Le «console application», che leggono e scrivono sullo standard input e output senza interfaccia grafica, hanno struttura simile

- Entrambi i linguaggi dispongono di apposite classi per input e ouput sulla console
  - Java: `System.out.println("testo")` oppure `System.in.read()`
  - C#: `Console.WriteLine("testo")` oppure `Console.Read()`

DIFFERENZE

- I dettagli delle classi e dei relativi metodi variano
- Le stringhe di formato usano sequence di escape differenti:
  - Java usa specificatori di formato simili al C (es. **`%d`**), ma estendibili a nuovi tipi mediante l'interfaccia `Formattable`
  - C# usa specificatori che hanno la forma **`{<indice>[,allineamento][:formato]}`** ed è possibile introdurre formati per nuovi tipi implementando l'interfaccia `IFormattable`

Console I/O – Esempio

```java
public static void main(String[] args)
{
  DataInput in = new DataInputStream(System.in);

  System.out.println("Come ti chiami?");
  String name = in.readLine();

  System.out.println("Quanti anni hai?");
  int age = Integer.parseInt(in.readLine());
  System.out.printf("%s ha %d anni", name, age);

  System.out.println("Valore intero a:");
  int a = Integer.parseInt(in.readLine());
  System.out.println("Valore intero b:");
  int b = Integer.parseInt(in.readLine());

  System.out.println("1) a+b\n2) a*b");
  int c = System.in.read();

  int r;
  switch (c){
    case '1': r = a+b; break;
    case '2': r = a*b; break;
    default: System.out.println("Err"); return;
  }

  System.out.printf("Risultato: %d", r);
 }
```

```csharp
public static void Main()
{
  Console.WriteLine("Come ti chiami?");
  string name = Console.ReadLine();

  Console.WriteLine("Quanti anni hai?");
  int age = int.Parse(Console.ReadLine());
  Console.WriteLine("{0} ha {1} anni", name, age);

  Console.WriteLine("Valore intero a:");
  int a = int.Parse(Console.ReadLine());
  Console.WriteLine("Valore intero b:");
  int b = int.Parse(Console.ReadLine());

  Console.WriteLine("1) a+b\n2) a*b");
  int c = Console.Read();

  int r;
  switch (c)
  {
    case '1': r = a + b; break;
    case '2': r = a * b; break;
    default: Console.WriteLine("Err"); return;
  }

  Console.WriteLine("Risultato: {0}", r);
}
```

Tipi di dato

ANALOGIE

- Tipi primitivi in Java e «tipi valore» predefiniti in C# (**`int`**, **`char`**, **`byte`**, **`double`**, **`bool`**, …)
  - Si noti che in C# il tipo valore relativo al tipo Boolean è definito con la keyword «**`bool`**»
- Tipi riferimento (reference type): **`object`**, **`string`**, array, classi, interfacce
  - Si noti che in C# il tipo riferimento per le stringhe è accessibile con la keyword «**`string`**» ('s' minuscola)
- Costanti: keyword «**`final`**» in Java, «**`const`**» e «**`readonly`**» in C#
- Cast e conversioni (es. conversione a stringa con metodo `ToString()`)

DIFFERENZE

- In C# non si usano classi «wrapper» per i tipi primitivi: i tipi valore predefiniti sono sempre visti come «derivati» da object, mediante boxing automatico
  - Si possono utilizzare sia le keyword del linguaggio (es. «**`object`**», «**`string`**», «**`int`**»), sia i nomi dei tipi corrispondenti nella class library (con la maiuscola es. «`Object`», «`String`», «`Int32`»).
- Enum – sono semplici tipi valore in C#, mentre sono vere e proprie classi in Java
  - In C# possono essere associati esplicitamente i valori integer associati ai mebri dell'enumerato

Tipi di dati – Esempi

JAVA

```java
// Tipi valore - Esempi di tipi valore predefiniti
boolean flag = true;
byte a = -128; // N.B. con segno

char c = 'e';
// Altri tipi valore predefiniti: short, int, long,
// float, double

// Tipi riferimento predefiniti
Object o = new Object();
String s = "Java";
// Tipi riferimento definibili dal programmatore:
// array, class, interface

// Costanti
final double Value = 42;

// Esempi di conversioni
int x = 42;
String y = Integer.toString(x);  // y = "42"
x = Integer.parseInt(y);
double z = 3.5;
x = (int) z;   // x = 3  (tronca i decimali)
// Esempi di enum
enum Action {START, STOP, REWIND, FORWARD};
Action a = Action.STOP;
if (a != Action.START)
  System.out.println(a);  // "Stop"
```

C#

```csharp
// Tipi valore - Esempi di tipi valore predefiniti
bool flag = true;
byte a = 255; // N.B. senza segno
sbyte b = -128; // N.B. con segno
char c = 'è';
// Altri tipi valore predefiniti: short, ushort, int, uint,
long,
// ulong, float, double, decimal

// Tipi riferimento predefiniti
object o = new object();
string s = "C#";
// Tipi riferimento definibili dal programmatore:
// array, class, interface, delegate

// Costanti
const double PI = 3.14; // iniz. a compile-time
readonly int Value = 42; // iniz. a runtime

// Esempi di conversioni
int x = 42;
string y = x.ToString();  // y = "42"
x = int.Parse(y);   // o x = Convert.ToInt32(y);
double z = 3.5;
x = (int) z;   // x = 3  (tronca i decimali)

// Esempi di enum
enum Action { START, STOP, REWIND, FORWARD };
Action a = Action.STOP;
if (a != Action.START)
  Console.WriteLine(a);  // "Stop"
```

Istruzioni: controllo di flusso e cicli

ANALOGIE

- «**if**», «**else**», «**else if**»: identici
- «**switch**»: analogo ma con alcune differenze rilevanti
- cicli «**for**», «**while**», «**do…while**»: identici
- ciclo «**foreach** (… **in** …) » in C#
  - corrisponde a «`for (… : …)` » in Java

DIFFERENZE

- switch: C# non consente il «passaggio» da un «**case**» al successivo omettendo il «**break**», ma è possibile esplicitamente trasferire il controllo a un qualsiasi «case» con la sintassi «**goto case** …».

```
switch (p)
{
  case 1:
    Console.Write("case 1");
    break;

  case 2:
    Console.Write("case 2");
    goto case 3;

  case 3:
    Console.Write("case 2/3");
    break;

  default:
    break;
}
```

```
switch(p)
{
  case 1:
    Console.Write("case 1");
    break;

  case 2:
  case 3:
    Console.Write("case 2/3");
    break;

  default:
    break;
}
```

```
switch(p)
{
  case 1:
    Console.Write("case 1");
    break;

  case 2:
    Console.Write("case 2");

  case 3:
    Console.Write("case 3");
    break;

  default:
    break;
}
```

Istruzioni: controllo di flusso e cicli – Esempi

```java
int j = 0;
while (j < 10){
  j++;
}

for (int i = 2; i <= 10; i += 2){
  System.out.println(i);
}

do{
  j++;
} while (j < 20);

for (int i : array){
  sum += i;
}

ArrayList<Object> list = new ArrayList<Object>();
list.Add(42);
list.Add("unibo");
list.Add(3.14);

for (Object o : list){
  System.out.println(o);
}
```

```csharp
int j = 0;
while (j < 10)
{
  j++;
}

for (int i = 2; i <= 10; i += 2)
{
  System.Console.WriteLine(i);
}

do
{
  j++;
} while (j < 20);

foreach (int i in array)
{
  sum += i;
}

ArrayList list = new ArrayList();
list.Add(42);
list.Add("unibo");
list.Add(3.14);

foreach (object o in list)
{
  System.Console.WriteLine(o);
}
```

Caratteri e stringhe

ANALOGIE

- Tipo char – caratteri con codifica UTF-16
- Classe string: sequenza di caratteri *immutabile* (qualsiasi operazione su stringhe crea una nuova stringa)
- Sequenze di escape in caratteri o stringhe (es. `'\0'`, `"\n\t\\"`, `'\u6C34'`, …)
- Concatenazione efficiente di stringhe mediante apposite classi
  - `StringBuilder` in C#, `StringBuffer` in Java

DIFFERENZE

- In C# il prefisso `@` consente di definire un «verbatim string literal»
  - tutti i caratteri che seguono sono «interpretati così come sono» (utile se si devono ignorare le sequenze di escape o scrivere stringhe multi-linea)
  - Es. `@"C:\temp\"` equivale a `"C:\\temp\\"`
- In C# gli operatori `==` e `!=` sono ridefiniti sulla classe string e confrontano i caratteri invece dei riferimenti
- In C# **`string`** implementa un indexer per cui è possibile accedere ai singoli caratteri con la sintassi degli array
  - Es. `char c1 = testo[i]` oppure `char c2 = "mio testo"[2]`

Caratteri e stringhe – Esempi

```java
char newLine = '\n';
int n = (int)newLine; // 13
char test = (char)13; // '\n'

// Concatenazione
String u = "Bologna";
u = "Universita di " + u; //"Universita di Bologna"

// Confronto
String c = "Cesena";
boolean e1 = c == "Cesena";  // Attenzione...
boolean e2 = c.equals("Cesena");   // true
boolean e3 = c.equalsIgnoreCase("CESENA"); // true
boolean e4 = c.compareTo("Cesena") == 0;   // true

// Formattazione date
Calendar d = new GregorianCalendar(1974, 11, 29);
String s = String.format("Data: %1$td/%1$tm/%1$tY", d);

// Modifica e concatenazione efficiente
StringBuffer buffer = new StringBuffer("due ");
buffer.append("tre ");
buffer.insert(0, "uno ");
buffer.replace(4, 7, "DUE");
System.out.println(buffer);   // "uno DUE tre"

for (char c : s.toCharArray()){ System.out.println(c); }
```

```dotnet
char newLine = '\n';
int n = (int)newLine; // 13
char test = (char)13; // '\n'

// Concatenazione
string u = "Bologna";
u = "Università di " + u; //"Università di Bologna"

// Confronto
string c = "Cesena";
bool e1 = c == "Cesena"; // confronta i caratteri
bool e2 = c.Equals("Cesena"); // true
bool e3 = c.Equals("Cesena", StringComparison.
    InvariantCultureIgnoreCase); //true
bool e4 = c.CompareTo("Cesena") == 0; // true

// Formattazione date
DateTime d = new DateTime(1974, 12, 29);
string s = string.Format("Data: {0:dd/MM/yyyy}", d);

// Modifica e concatenazione efficiente
StringBuilder buffer = new StringBuilder("due ");
buffer.Append("tre ");
buffer.Insert(0, "uno ");
buffer.Replace("due", "DUE");
Console.WriteLine(buffer); // "uno DUE tre"

foreach (char c in s) { Console.WriteLine(c); }
```

Operatori

ANALOGIE

- Java e C# condividono i principali operatori
  - +, - , *, /, ++, --, &&, ||, ?:, ==, =,

DIFFERENZE

- L'operatore Java >>> (unsigned right shift) non esiste in C#
  - non è necessario poiché il C# supporta variabili intere unsigned
- L'operatore C# corrispondente a «`instanceof`» di Java è «**`is`**»

Java	C#	Description
`x.y`	`x.y`	Member access "dot" operator
`f(x)`	`f(x)`	Method invocation operator
`a[x]`	`a[x]`	Array element access operator
`++, --`	`++, --`	Increment and decrement operators (pre and post-fix)
`new`	`new`	Object instantiation operator
`instanceof`	`is`	Type verification operator
`(T)x`	`(T)x`	Explicit cast operator
`+, -`	`+, -`	Addition and subtraction operators (binary). Positive and negative operators (unary)
`+`	`+`	String concatenation operator
`!`	`!`	Logical negation operator
`&&, ||`	`&&, ||`	Conditional AND and OR operators (short-circuited evaluation)
`&, |, ^`	*n/a*	Conditional AND, OR, and XOR operators (full evaluation of operands)
`~`	`~`	Bitwise complement operator
`&, |, ^`	`&, |, ^`	Bitwise AND, OR, and XOR operators
`<<, >>`	*n/a*	Signed left-shift and right-shift operators
`>>>`	`>>`	Unsigned right-shift operator
`*, /, %`	`*, /, %`	Multiply, divide, and modulus operators
`==, !=`	`==, !=`	Is-equal-to and is-not-equal-to operators
`<, >, <=, >=`	`<, >, <=, >=`	Relational less-than, greater-than, less-than-or-equal-to, and greater-than-or-equal-to operators
`x?y:z`	`x?y:z`	Conditional operator
`=`	`=`	Assignment operator

Array

ANALOGIE

- Ogni array è un oggetto (reference type), con lunghezza fissa
  - Sintassi simile al C/C++ per accedere ai valori, con controllo validità dell'indice a runtime.
- Stessa sintassi per allocare un array, con possibilità di specificare l'elenco dei valori (array initializer)
  - se si specifica solo la dimensione, c'è un valore di default predefinito per tutti gli elementi (0, false, null).
- Si possono creare jagged array (ovvero, array di array)
- Classe contenente i metodi di utilità per gli array (es. sort, ricerca binaria, copia elementi, …)
  - `Array` in C#, **`Arrays`** in Java.

DIFFERENZE

- C# supporta array multidimensionali, oltre agli array jagged, mediante la sintassi **`[,]`**
  - Es. `int[,] matrix = new int[5,8];`

- In C# qualsiasi classe può definire un «indexer» per utilizzare la stessa sintassi degli array per l'accesso a suoi element

Array – Esempi

```java
int[] v = new int[] { 5, 4, 3, 2, 1 };
for (int i = 0; i < v.length; i++){
  System.out.println(v[i]);
}

String[] nomi = new String[5];
nomi[0] = "James Gosling";

float[][] mat = new float[][]{
    new float[] {1,2,3}, new float[]{5,8,13}};
mat[1][0] = 4.2f;

int rows = mat.length; // 2 (numero righe)
int cols = mat[0].length; // 3 (numero colonne)

int[][] jagged = new int[][]{
                    new int[5], new int[2], new int[3]};
jagged[0][4] = 5;

int[] v2 = v;  // N.B. non crea una copia
int[] v3 = (int[])v.clone(); // Crea una copia

Arrays.sort(v); // 1, 2, 3, 4, 5
System.out.println(Arrays.toString(v));
int j = Arrays.binarySearch(v, 2);  // 1
```

```csharp
int[] v = new int[] { 5, 4, 3, 2, 1 };
for (int i = 0; i < v.Length; i++)
{
  Console.WriteLine(v[i]);
}

string[] nomi = new string[5];
nomi[0] = "Anders Hejlsberg";

float[,] mat = new float[,]{{1,2,3},{5,8,13}};
mat[1, 0] = 4.2f;

int r = mat.Rank; // 2 (numero di dimensioni)
int tot = mat.Length; // 6 (totale elementi)
int rows = mat.GetLength(0); // 2 (numero righe)
int cols = mat.GetLength(1); // 3 (numero colonne)

int[][] jagged = new int[3][]{
                    new int[5], new int[2], new int[3]};
jagged[0][4] = 5;

int[] v2 = v;  // N.B. non crea una copia
int[] v3 = (int[])v.Clone(); // Crea una copia

Array.Sort(v); // 1, 2, 3, 4, 5
Console.WriteLine("[{0}]", string.Join(",", v));
int j = Array.BinarySearch(v, 2);  // 1
Array.Reverse(v); // 1, 2, 3, 4, 5
```

Variabili locali e array implicitamente tipizzati

- Le variabili locali possono avere un tipo *dedotto al momento dell'inizializzazione* anziché un tipo esplicito (type inference)
  - Introdotta dalla versione 3.0 del linguaggio
- Si utilizza la parola chiave **`var`**
  - Si indica al compilatore di inferire il tipo della variabile dall'espressione sul lato destro dell'istruzione di inizializzazione
  - Nota: Il tipo è determinato a **compile-time**, non a run-time...
- Questo meccanismo consente al linguaggio di aderire meglio al principio DYR («Don't Repeat Yourself»)
- N.B.: Java supporta la medesima funzionalità dalla versione 10

```
int i = 123;
string s = "Hello world";
double d = 10.3;
UnaClasse r = new UnaClasse();
int[] v = new int[] {1, 2, 3};
```

=

```
var i = 123;
var s = "Hello world";
var d = 10.3;
var r = new UnaClasse();
var v = new int[] {1, 2, 3};
```

Classi

ANALOGIE

- In generale sintassi identica o molto simile
- Controllo accesso: «**`public`**» e «**`private`**» hanno lo stesso significato («**`protected`**» no!, vedi sotto).
- Possibilità di definire campi e metodi «**`static`**»
- Possibilità di richiamare un costruttore da un altro costruttore (con sintassi leggermente diversa)

DIFFERENZE

- In C# esiste il modificatore «**`internal`**» che rende l'elemento accessibile solo all'interno dell'assembly in cui è definito
- In Java «**`protected`**» consente l'accesso sia da classi derivate che da quelle nello stesso package, in C# solo da classi derivate.
  - In C# esiste «**`protected internal`**», cioè «**`internal`**» dentro l'assembly, «**`protected`**» fuori da esso
- La visibilità di default (se non si specifica un modificatore) in Java è «tutto il package».
  - In C# invece l'accesso di default è «**`private`**».
- Attenzione alla keyword «**`static`**» applicata a una classe
  - in C# indica che una classe ha solo membri «**`static`**»

Classi – Esempio

```java
class Counter
{
 // Il campo è reso inaccessibile direttamente
 private int value;

 // E' il costruttore che inizializza i campi!
 public Counter()
 {
   this(0); // Riusa l'altro costruttore
 }

 // Un costruttore con parametro
 public Counter(int value)
 {
   this.value = value;
 }

 // Unico modo per osservare lo stato
 public int getValue()
 {
    return this.value;
 }

 // Unico modo per modificare lo stato
 public void inc()
 {
    this.value++;
 }
}
```

```csharp
class Counter
{
  // Il campo è reso inaccessibile direttamente
  private int value;

  // E' il costruttore che inizializza i campi!
  public Counter() : this(0) // Riusa l'altro costruttore
  {
  }

  // Un costruttore con parametro
  public Counter(int value)
  {
    this.value = value;
  }

  // Unico modo per osservare lo stato
  // N.B. in C# sarebbe più opportuna una proprità
  public int GetValue()
  {
    return this.value;
  }

  // Unico modo per modificare lo stato
  public void Inc()
  {
    this.value++;
  }
}
```

Classi Astratte

- È possibile dichiarare una classe come **abstract**
  - Classi astratte non possono essere istanziate (non si può creare un oggetto di tali classi)
  - Sono tipicamente utilizzate per fornire funzionalità comuni a una serie di classi derivate
  - Contengono tipicamente uno o più metodi **abstract** (ovvero metodi virtuali senza implementazione)

```
abstract class FormaGeometricaPiana
{
   public abstract int CalcolaPerimetro();
}

class Quadrato : FormaGeometricaPiana
{
   public override int CalcolaPerimetro()
   {
      return 0;
   }
}
```

Passaggio dei parametri

ANALOGIE

- Passaggio per valore: identico.
- Metodi con numero variabile di argomenti
  - visti come un parametro di tipo array all'interno del metodo
  - «`void m(int… args)`» in Java è equivalente a «`void m(params int[] args)`» in C#

DIFFERENZE

- C#: supporta anche il passaggio esplicito per riferimento (mediante le parole chiave «**`ref`**» oppure «**`out`**»)
  - I parametri contrassegnati con «**`out`**» devono essere assegnati obbligatoriamente prima del **`return`** del metodo.
- Il C# consente di definire parametri opzionali (con un valore di default) e di identificare esplicitamente gli argomenti per nome al momento della chiamata.
- In Java è possibile dichiarare un parametro come «`final`», per indicare che il suo valore non sarà modificato dal metodo: non è prevista una cosa analoga in C#

Passaggio dei parametri – Alcuni Esempi

```
partial class Program
{
  static void M1(int n)
  {
    n++;
    Console.WriteLine("N inside m1 = " + n);
  }

  static void M2(ref int n)
  {
    n++;
    Console.WriteLine("N inside m2 = " + n);
  }

  static void M3(out int a)
  {
    a = 10;
    Console.WriteLine("P inside M3 = " + a);
  }

  static void M4(params int[] v)
  {
  }

  static int Sum(int x, int y = 0)
  {
    return x + y;
  }
}
```

```
partial class Program
{
  public static void Main()
  {
    int n = 0;
    Console.WriteLine("N inside Main() = " + n);
    Console.WriteLine("Calling M1()");
    M1(n);
    Console.WriteLine("N inside Main() = " + n);
    Console.WriteLine("Calling M2()");
    M2(ref n);
    Console.WriteLine("N inside Main() = " + n);

    int p;
    Console.WriteLine("Calling M3()");
    M3(out p);
    Console.WriteLine("P inside Main() = " + p);

    M4(1, 2, 3);

    int s1 = Sum(1, 2);
    int s2 = Sum(5);
  }
}
```

C:\WINDOWS\system32\cmd.exe

```
N inside Main() = 0
Calling M1()
N inside m1 = 1
N inside Main() = 0
Calling M2()
N inside m2 = 1
N inside Main() = 1
Calling M3()
P inside M3 = 10
P inside Main() = 10
```

Eccezioni

ANALOGIE

- Stessa sintassi:
  - «**try**»…«**catch**»…«**finally**», «**throw**»
- Gerarchia di ereditarietà delle eccezioni: derivano tutte da una classe «`System.Exception`»

```
try
{
    //Statements which may throw an exception
    throw new Exception();
}
catch (Exception e)
{
    //Exception handling
}
finally
{
    //Clean-up code
}
```

DIFFERENZE

- In C# non sono previste le «checked exceptions»
  - non vi è quindi un analogo della keyword «**throws**»
- In C# non sono ammesse istruzione di control-passing nel blocco **finally** (es. **break** o **return**).

Eccezioni – Esempio

```java
class DeviceIsOverException extends Exception{ }

...

// ...AbstractDevice
public void on() throws DeviceIsOverException{
  if (!this.on) {
    throw new DeviceIsOverException(this);
  }
}

// ...DeviceRow
public void allOn() throws DeviceIsOverException{
  for (Device d : this.list){
    try {
      d.on();
    } catch (DeviceIsOverException de) {
      // do some stuff!
    }
  }
}
```

```dotnet
class DeviceIsOverException : Exception {}

...

// ...AbstractDevice
public void on()
{
  if (!this.on)
  {
    throw new DeviceIsOverException(this);
  }
}

// ...DeviceRow
public void allOn()
{
  foreach (IDevice d in this.list)
  {
    try
    {
      d.on();
    }
    catch (DeviceIsOverException de)
    {
      // do some stuff!
    }
  }
}
```

Metadati/annotazioni

ANALOGIE

- Sia Java che C# consentono di annotare il codice con metadati per estendere le funzionalità del linguaggio di programmazione e le potenzialità delle applicazioni a runtime
  - in C# si utilizzano gli «attributi», in Java le «annotazioni».
- In entrambi i casi alcuni tipi di annotazioni sono già disponibili ed è possibile definire nuovi tipi di annotazioni
- Un esempio fra i molteplici impieghi di tali metadati
  - in entrambi i linguaggi sono utilizzati per annotare le classi e i metodi di test nei più diffusi framework di «unit testing»

DIFFERENZE

- La sintassi è differente: `[Attribute]` in C# e `@Annotation` in Java)

- Il C# supporta anche alcune direttive per il compilatore che si possono inserire nel codice sorgente in modo simile al C/C++ (es. `#pragma`, `#region`…`#endregion`, …)

Metadati/annotazioni – Esempio

```java
class A
{
  protected void m(int x){}
}

class B extends A
{
  @Override
  protected void m(int x){}

  @Override
  protected void m(string x){} // comp. error

  @Deprecated
  public void old() { }  // comp. warning se usato
}

...

static void deprec()
{
  new B().old(); // genera il warning
}

@SuppressWarnings("deprecation")
static void noDeprec()
{
  new B().old(); // non genera il warning
}
```

```csharp
class A
{
  protected virtual void m(int x) { }
}

class B : A
{
  override // N.B. in C# è una keyword obbligatoria
  protected void m(int x) { }

  override
  protected void m(string x) { } // comp. error

  [Obsolete]
  public void Old() { }  // comp. warning se usato
}

...
static void deprec()
{
  new B().Old(); // genera il warning
}

#pragma warning disable 0612, 0618
static void noDeprec()
{
  new B().Old(); // non genera il warning
}
#pragma warning restore 0612, 0618
```

Generazione di documentazione dai commenti

ANALOGIE

- Sia Java che C# forniscono un meccanismo per estrarre commenti appositamente formattati dal codice sorgente e inserirli in automatico in un documento separato.

- Questi commenti sono tipicamente utilizzati per generare la documentazione del codice sorgente
  - API Reference

DIFFERENZE

- In Java i commenti di documentazione sono delimitati da `/** … */`
  - il contenuto è in formato HTML con speciali tag (@param, @return, …).
  - Il tool «**javadoc**» produce una documentazione HTML a partire da tali commenti.
- In C# sono espressi con `///` su ogni riga (caso più comune) o delimitati da `/** … */` (come in Java)
  - il contenuto è in formato XML con appositi tag predefiniti (`<param>`, `<returns>`, …).
  - Il compilatore C# produce un documento XML a partire da tali commenti, che può poi essere utilizzato dallo stesso ambiente di sviluppo (es. per visualizzare tooltip nell'editor dei sorgenti con la descrizione di classi e metodi), o da appositi tool per produrre documentazione.

Generazione di documentazione dai commenti

```java
/**
 * Class level summary documentation...
 * <p>Longer comments...
 * <p>other comments...
 * <p>other comments...
 */
class SomeClass
{
  /** The class constructor. */
  public SomeClass() { }

  /** Desc. for SomeMethod.
   *  @param s Description for s
   *  @see String Reference to a type or
   *  member */
  public void SomeMethod(String s) { }

  /** Some other method.
   *  @return Return results descr.
   *  @link #SomeMethod(string)
   *  Notice the use of the @link to
   *  reference a specific method */
  public int SomeOtherMethod() { return 0; }

  /**
   *  The entry point for the application.
   *
   *  @param args A list of command line
   *  arguments */
  public static void main(String[] args)
  {
  }
}
```

```csharp
/// <summary>
/// Class level summary documentation...</summary>
/// <remarks>Longer comments...
/// <para>other comments...</para>
/// <para>other comments...</para>
/// </remarks>
public class SomeClass
{
  /// <summary>The class constructor.</summary>
  public SomeClass() { }

  /// <summary>Desc. for SomeMethod.</summary>
  /// <param name="s">Description for s</param>
  /// <seealso cref="String">Reference to a type or
  /// member.</seealso>
  public void SomeMethod(string s) { }

  /// <summary>Some other method.</summary>
  /// <returns>Return results descr.</returns>
  /// <seealso cref="SomeMethod(string)">
  /// Notice the use of the cref attribute to
  /// reference a specific method </seealso>
  public int SomeOtherMethod() { return 0; }

  /// <summary>
  /// The entry point for the application.
  /// </summary>
  /// <param name="args"> A list of command line
  /// arguments</param>
  public static void Main(string[] args)
  {
  }
}
```

Reflection

ANALOGIE

- Entrambi i linguaggi forniscono un meccanismo di «reflection», ossia la possibilità di identificare campi e metodi di una classe a runtime, di invocarne i metodi, costruirne nuove istanze, etc.

- Un'apposita classe rappresenta i tipi
  - «`Class<T>`» in Java e «`Type`» in C#
  - Ogni istanza di tale classe è un tipo (es. una certa interfaccia o una certa classe).
  - Altre classi rappresentano metodi, costruttori, etc.

DIFFERENZE

- In C# la reflection avviene a livello di «assembly», mentre in Java a livello di «class»

- In Java l'istanza di `Class<T>` corrispondente a un determinato tipo si ottiene con «`.class`» (es. «`String.class`»), mentre in C# con «`typeof()`» (es. «`typeof(string)`»)

Reflection – Esempio

```java
import java.io.*;
import java.lang.reflect.*;

class DynamicExecution{
  final static String Q_CLASS = "Nome classe: ";
  final static String Q_METH = "Nome metodo: ";
  final static String L_OK = "OK: result = ";
  final static String E_RET = "Errore tipo ritorno";

  public static void main(String[] s) throws Exception{
    BufferedReader reader = new BufferedReader(
                 new InputStreamReader(System.in));
    System.out.println(Q_CLASS);
    String className = reader.readLine();
    System.out.println(Q_METH);
    String methName = reader.readLine();
    Class<?> cl = Class.forName(className);
    Constructor<?> cns = cl.getConstructor();

    Method met = cl.getDeclaredMethod(methName);
    Class<?> rt = met.getReturnType();
    if (!rt.isAssignableFrom(String.class)) {
      throw new NoSuchMethodException(E_RET);
    }

    Object o = cns.newInstance();
    String result = (String)met.invoke(o);
    System.out.println(L_OK + result);
  }
}
```

```dotnet
using System;
using System.Reflection;

class DynamicExecution
{
  readonly static string Q_CLASS = "Nome classe: ";
  readonly static string Q_METH = "Nome metodo: ";
  readonly static string L_OK = "OK: result = ";
  readonly static string E_RET = "Errore tipo di ritorno";

  public static void Main()
  {
    Console.WriteLine(Q_CLASS);
    string className = Console.ReadLine();
    Console.WriteLine(Q_METH);
    string methName = Console.ReadLine();
    Type cl = Type.GetType(className);
    ConstructorInfo cns =
                cl.GetConstructor(Type.EmptyTypes);
    MethodInfo met = cl.GetMethod(methName);
    Type rt = met.ReturnType;
    if (!rt.IsAssignableFrom(typeof(string)))
    {
      throw new InvalidOperationException(E_RET);
    }
    object o = cns.Invoke(null);
    string result = (string)met.Invoke(o,null);
    Console.WriteLine(L_OK + result);
  }
}
```

PARTE 2

Proprietà e Object Initializers

Tipi valore (struct)

- Simili alle classi ma con importanti differenze
  - Sono tipi valore: le loro variabili non sono riferimenti ma contengono direttamente il loro valore!
  - Non hanno ereditarietà
  - Possono contenere metodi e proprietà e costruttori
  - I campi non possono essere inizializzati al momento della dichiarazione

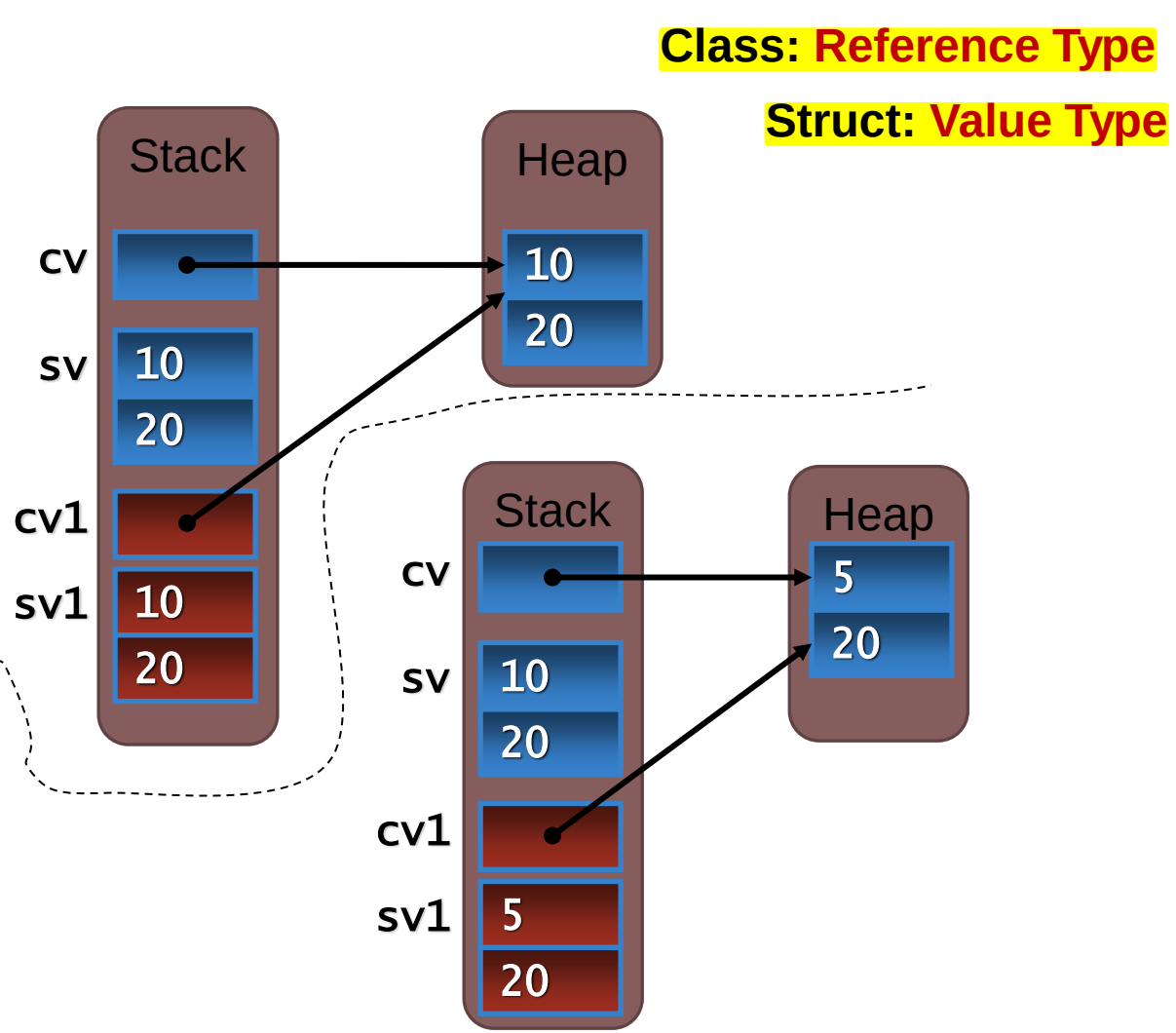
```
struct Vector
{
    private double vx, vy;

    public Vector(double vx,double vy)
    {
        this.vx = vx;
        this.vy = vy;
    }

    public double CalculateNorm()
    {
        return Math.Sqrt(vx*vx + vy*vy);
    }
}
```

- Ideali per oggetti “leggeri”
  - Sono allocate nello stack (a differenza degli oggetti che sono sempre allocati nello heap) e non richiedono GC

```
Vector P; //Alloca la struttura senza new: i campi
          //sono inizializzati ai valori di default

Vector P1 = new Vector(2,10);
          // Alloca la struttura chiamando un
          //costruttore con parametri

double n = P1.CalculateNorm();
```

Struct vs. Classi

```
class CVector
{ public double vx, vy; ... }

struct SVector
{ public double vx, vy; ... }

CVector cv = new CVector(10,20);

SVector sv = new SVector(10,20);

CVector cv1 = cv;

SVector sv1 = sv;

cv1.vx = 5;
sv1.vx = 5;

Console.WriteLine(sv.vx); // 10
Console.WriteLine(cv.vx); // 5!
```



Proprietà

Coniugano la semplicità dei campi con la flessibilità dei metodi

- Vi si accede come se fossero campi (es. `if(counter.Value > 42)`) ma in realtà sono metodi
  - **`get`** : chiamato quando viene letto il valore della proprietà.
  - **`set`** : chiamato quando viene assegnato un nuovo valore alla proprietà
    - la keyword «**`value`**» referenzia il valore
    - se **`set`** non è specificata, la proprietà è read-only.

- Nei casi in cui in Java si implementano metodi *getter/setter*, in C# di solito si usano le *proprietà*.

```
public class Counter
{
  protected int value;
  ...
  public int GetValue()
  {
     return this.value;
  }
}
```

```
public class Counter
{
  protected int value;
  ...
  public int Value
  {
     get { return this.value; }
  }
}
```

ESEMPIO

```
class Person
{
  private int age = 0;

  public int Age
  {
    get { return age; }
    set { age = value; }
  }
}

...

Person p = new Person();

// Esempio di chiamata set
p.Age = 42;

// Esempio di chiamata get
int x = p.Age;

// Esempio di chiamata get e set
p.Age += 1;
```

Proprietà - Esempio con Dato Derivato

```
class Person
{
  private string name, surname;
  private DateTime dateOfBirth;

  public int Age // Proprietà di tipo int (read-only)
  {
    get
    {
      DateTime now = DateTime.Now;
      DateTime dn = dateOfBirth;

      int age = now.Year - dn.Year;

      if (now.Month < dn.Month || (now.Month == dn.Month && now.Day < dn.Day))
      {
        age--;
      }

      return age;
    }
  }
...

}
```

```
static void Main()
{
  DateTime dob = ...
  Person p1 = new Person("Luca","Bianchi",dob);
  Console.WriteLine(p1.Age);
}
```

Proprietà automatiche

- Proprietà auto-implementate
  - Disponibili dalla versione 3.0 del linguaggio
- Permettono di rendere la dichiarazione delle proprietà più semplice e concisa.
- Utili per proprietà che semplicemente permettono l'accesso a un membro privato, senza implementare logiche specifiche nei metodi di accesso (get/set).
  - Il compilatore genera automaticamente un campo privato collegato alla proprietà

Codice Originale

```
class Persona
{
  public string Nome { get; set; }
  public string Cognome { get; set; }
  public DateTime DataDiNascita { get; private set; }
}
```

Codice «generato» dal Compilatore

```
class Persona
{
    private string nome;
    private string cognome;
    private DateTime dataDiNascita;

    public string Nome
    {
       get { return nome; }
       set { nome = value; }
    }

    public string Cognome
    {
       get { return cognome; }
       set { cognome = value; }
    }

    public DateTime DataDiNascita
    {
       get { return dataDiNascita; }
    }
}
```

ESEMPIO

1-Properties

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Properties
{
    class MainProperties
    {
        static void Main(string[] args)
        {
            Person mario = new Person("Mario", "Rossi", new DateTime(1994, 1, 4));

            Console.WriteLine(mario);
            Console.ReadKey();

            //giovanni.Age = 25;

            //Console.WriteLine(mario);
            //Console.ReadKey();

            //Person luigi = new Person() {
            //    FullName = "Luigi Bianchi",
            //    BirthDate = new DateTime(1992, 1, 4)
            //};

            //Console.WriteLine(luigi);
            //Console.ReadKey();
        }
    }
}
```

Object Initializers

- Permettono di assegnare un valore a proprietà e campi pubblici di una classe al momento della creazione di una sua istanza
- Senza che vi sia un costruttore che esplicitamente accetta tali parametri

```
class Persona
{
  public string Nome { get; set; }
  public string Cognome { get; set; }
}
```

```
var p1 = new Persona();
p1.Nome = "Pluto";

var p2 = new Persona();
p2.Nome = "Mario";
p2.Cognome = "Rossi";
```

```
var p1 = new Persona() {
            Nome = "Pluto"
        };

var p2 = new Persona() {
            Nome = "Mario",
            Cognome="Rossi"
        };
```

- Possono essere «emulati» in Java combinando:

  Classi Aninome + Initializzatori

- Ma funziona solo se:
  - Ci si riferisce a campi/metodi protetti o pubblici
  - La classe NON è final

```
public class Persona {
  private String nome;
  private String cognome;

  public void setNome(String nome) {
    this.nome = nome;
  }

  public void setCognome(String cognome) {
    this.cognome= cognome;
  }
}
```

```
Persona p1 = new Persona() {{
  setNome("Mario");
  SetCognome("Rossi")
}};
```

Collection Initializers

- Permettono di specificare gli elementi iniziali di una struttura dati

```
var nums = new List<int> { 0,1,2,3 };
```

```
var dict = new Dictionary<int, string>()
{
  { 1, "First" },
  { 2, "Second" },
  { 3, "Third" }
};
```

```
var dict = new Dictionary<int, string>()
{
  [1] = "First",
  [2] = "Second",
  [3] = "Third"
};
```

- Disponibili a partire da Java 9 i seguenti metodi statici:

```
static <E> List<E> List.of()
static <E> List<E> List.of(E... elements)

static <E> Set<E> Set.of()
static <E> Set<E> Set.of(E... elements)

static <K, V> Map<K, V> Map.of()
static <K, V> Map<K, V> Map.of(K key, V val)
static <K, V> Map<K, V> Map.of(K k1, V v1,
                               K k2, V v2)
...
```

- Restituiscono collezioni immutabili!

PARTE 3

Ereditarietà, Polimorfismo, Interfacce, Eccezioni, Dynamic Binding

Ereditarietà

ANALOGIE

- Eredità singola
  - riferimento a classe padre: «**`super`**» in Java, «**`base`**» in C#
- Polimorfismo con late binding
- Classi/metodi virtuali non estendibili/ridefinibili: «**`final`**» in Java, «**`sealed`**» in C#

DIFFERENZE

- In Java tutti i metodi sono virtuali, mentre in C# (come in C++) sono virtuali solo i metodi definiti con la keyword «**`virtual`**».
- In C# nel ridefinire un metodo virtuale è necessario usare la keyword «**`override`**»
  - Analogie con l'annotazione Java **`@Override`**
- In C# è possibile ridefinire anche metodi non virtuali (con tutti i limiti del caso) tramite la keyword «**`new`**».
- In C# **non** si può cambiare visibilità nel ridefinire un metodo virtuale (es. da «**`protected`**» a «**`public`**»).

Ereditarietà – Esempio

```java
public class ExtendibleCounter{
  protected int value;

  public ExtendibleCounter(int initialValue){
    this.value = initialValue;
  }

  public void increment() {
    this.value++;
  }

  public int getValue() {
    return this.value;
  }
}
```

```java
public class LimitCounter extends ExtendibleCounter{
  protected int limit;

  public LimitCounter(int initialValue,int limit){
    super(initialValue);
    this.limit = limit;
  }

  public boolean isOver() {
    return value == limit;
  }

  // Overriding del metodo increment()
  public void increment(){
    if (!this.isOver())
      super.increment(); // richiamo versione padre
  }
}
```

```csharp
public class ExtendibleCounter
{
  protected int value;

  public ExtendibleCounter(int initialValue)
  {
    this.value = initialValue;
  }

  public virtual void Increment()
  {
    this.value++;
  }

  public int GetValue() {
    return this.value;
  }
}
```

```csharp
public class LimitCounter : ExtendibleCounter
{
  protected int limit;

  public LimitCounter(int initialValue,int limit)
     : base(initialValue)
  {
    this.limit = limit;
  }

  public bool IsOver()
  {
    return value == limit;
  }

  // Overriding del metodo increment()
  public override void Increment()
  {
    if (!this.IsOver())
      base.Increment();// richiamo versione base
  }
}
```

Overriding – Aspetti critici

- In Java tutti i metodi sono virtuali
  - A parte quelli etichettati come «**`final`**»

- In C# è il contrario: tutti i metodi sono finali
  - A parte quelli etichettati con «**`virtual`**»

- Quando il programmatore C# progetta una classe deve chiedersi quali metodi vuole che siano virtuali

- È importante evitare di cadere nel tranello "tanto c'è la «**`new`**»" nel caso in cui serva ridefinire un metodo non-virutale
  - Le keyword «**`new`**» ed «**`override`**» sembra facciano la stessa cosa ma non è così (vedasi esempio nella prossima slide)
  - In particolare «**`new`**» ha un comportamento subdolo e inusuale, che non ha equivalenti in Java

- I metodi etichettati con «**`new`**» non sono soggetti a *late-binding*

Polimorfismo e metodi virtuali – Esempi

```
class Persona
{
  public void StampaDesc()
  {
      Console.WriteLine("Person");
  }
}

class Studente : Persona
{
  public new void StampaDesc()
  {
      Console.WriteLine("Student");
  }
}
```

```
class Persona
{
  public virtual void StampaDesc()
  {
      Console.WriteLine("Person");
  }
}

class Studente : Persona
{
  public override void StampaDesc()
  {
      Console.WriteLine("Student");
  }
}
```

```
Persona A = new Persona();
Studente B = new Studente();
Persona C = B;
A.StampaDesc(); // Person
B.StampaDesc(); // Student
C.StampaDesc(); // Person
```

```
Persona A = new Persona();
Studente B = new Studente();
Persona C = B;
A.StampaDesc(); // Person
B.StampaDesc(); // Student
C.StampaDesc(); // Student
```

Type-checking & Casting

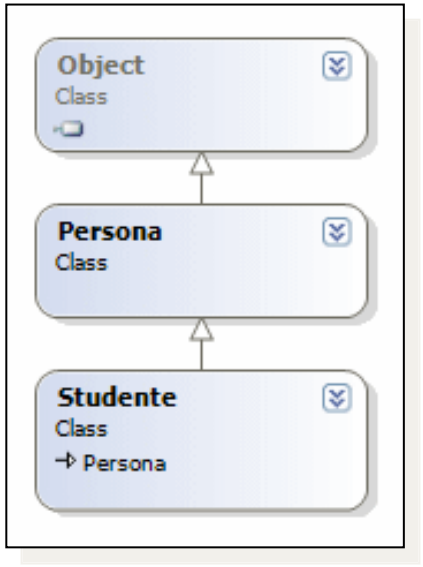
Cast

- Type cast (sintassi simile al C)
  - Se la conversione non può essere eseguita → errore (eccezione)
- Operatore **`as`**
  - Se il cast non può essere eseguito, il risultato è **`null`**

Type checking

- Operatore **`is`**
  - Permette di controllare se un oggetto appartiene a un determinato tipo

Type-checking & Casting – Esempio




```
Persona A = new Persona();
Studente B = new Studente();

Persona C = B; // Corretto: non richiede cast

Studente D = (Studente) C; // Corretto ma richiede cast
Studente E = (Studente) A; // Genera errore a run-time

Studente F = C as Studente; // Corretto: F=C
Studente G = A as Studente; // Corretto: G=null

bool b1 = (A is Persona) && (B is Studente); // true
bool b2 = F is Studente; // true
bool b3 = A is Studente; // false
bool b4 = G is Studente; // false
bool b5 = B is Object; // true
```

Interfacce

ANALOGIE

- Stesso significato, sintassi e utilizzo simile.
  - Al posto della keyword «`implements`» (utilizzata in Java) si utilizza (in C#) la stessa notazione («`:`») utilizzata per l'ereditarietà
- Una classe può implementare più interfacce.
- I membri delle interfacce sono implicitamente «public»
- Da C# 8 su runtime .NET Core 3.1 e successive sono supportati anche:
  - Dichiarazione di costanti, metodi statici e metodi di default.
  - Dichiarazione di classi e interfacce innestate

DIFFERENZE

- In C#, per convenzione, i nomi delle interfacce hanno il prefisso «`I`»
  - questo facilita la distinzione fra classe base e interfacce implementate dato che la sintassi è la stessa
- In C#, ci sono casi particolari (e.g. type checking su tipi generici) in cui interfacce e classi si comportano diversamente
  - poter distinguere a colpo d'occhio il nome di un'interfaccia è dunque utile in C#, mentre è inutile in Java
- In C# un'interfaccia, oltre a metodi, può contenere proprietà

Interfacce (2)

```java
public interface Device {
  void switchOn();

  void switchOff();

  boolean isSwitchedOn();
}

public class Lamp implements Device {
  public Lamp() { ... }

  public void switchOn() {
    this.switchedOn = true;
  }

  public void switchOff() {
    this.switchedOn = false;
  }

  public boolean isSwitchedOn() {
    return this.switchedOn;
  }
}
```

```csharp
public interface IDevice
{
  void SwitchOn();
  void SwitchOff();
  bool IsSwitchedOn();
}

public class Lamp : IDevice
{
  public Lamp() { ... }
  public void SwitchOn()
  {
    this.switchedOn = true;
  }
  public void SwitchOff()
  {
    this.switchedOn = false;
  }
  public bool IsSwitchedOn()
  {
    return this.switchedOn;
  }
}
```

Dynamic late binding

Tipo di dato «**dynamic**»

- Il compilatore assume che variabili di tale tipo supportino qualsiasi operazione (metodo, proprietà, operatore, …)
- A compile-time viene codificata opportunamente l'operazione che deve essere eseguita sull'oggetto e i relativi argomenti; a runtime se l'operazione non è possibile viene generata un'eccezione.
- Consente di semplificare l'utilizzo di oggetti di tipo sconosciuto come quelli ottenuti da linguaggi dinamici o mediante reflection.

```
string str = "Prova dynamic"; // string è un normale tipo non dynamic
Console.WriteLine(str.ToUpper()); // il compilatore conosce questo medoto
Console.WriteLine(str.MetodoInesistente()); // il compilatore qui darebbe errore


dynamic dyn = str; // dyn è una variabile di tipo dynamic
Console.WriteLine(dyn.ToUpper()); // compilabile e a runtime chiamerà string.ToUpper()
Console.WriteLine(dyn.MetodoInesistente()); // compilabile, ma a runtime darà eccezione
```

Esempio: Dynamic Dispatch

```
class A
{
  public virtual string GetValue()
  {
    return "a";
  }
}
```

```
class B : A
{
  public override string GetValue()
  {
    return "b";
  }
}
```

```
public static void Print(A x)
{
  Console.WriteLine("A: " + x.GetValue());
}
public void Print(B x)
{
  Console.WriteLine("B: " + x.GetValue());
}
```

```
A b1 = new B();
dynamic b2 = new B();

Print(b1); // A: b
Print(b2); // B: b
```

ESEMPIO

2-Dynamic Dispatch

```
using System;

namespace LezioneCSharp
{

    class A
    {
        public virtual string GetValue()
        {
            return "a";
        }
    }

    class B : A
    {
        public override string GetValue()
        {
            return "b";
        }
    }
    static class MainDynamicDispatch
    {
        static void Print(A x)
        {
            Console.WriteLine("A: " + x.GetValue());
        }

        static void Print(B x)
        {
            Console.WriteLine("B: " + x.GetValue());
        }

        static void Main(string[] args)
```

PARTE 4

Indicizzatori, Overloading di Operatori, Metodi Estensione

Indexers

- Un indexer permette di accedere a attributi interni di un oggetto come se fosse un array
  - Dichiarazione simile a quella delle proprietà (**`get`**/**`set`**)
  - Si utilizza la keyword **`this`** come nome della proprietà
  - È possibile utilizzare indexer mono- o multi-dimensionali (come per gli array)
  - Gli indici possono avere tipo qualsiasi, non solo **`int`**

```
public class Container
{
  public ElementType this[KeyType key]
  {
    get { /* usa key per recuperare un elemento */ }
    set { /* indicizza l'elemento value tramite key */ }
  }

}
```

Indexers – Esempio

Versione senza Indexer

```
public class DomusController
{
  private IDevice[] devices;
  ...

  public IDevice GetDevice(int pos)
  {
    return this.devices[pos];
  }

  public void SetDevice(int pos, IDevice dev)
  {
    this.devices[pos] = dev;
  }
}
```

Versione con Indexer

```
public class DomusController
{
  private IDevice[] devices;
  ...

  public IDevice this[int pos]
  {
    get { return devices[pos]; }
    set { devices[pos] = value; }
  }


}
```

```
var dc = new DomusController(10);

dc.SetDevice(0, new Lamp());
dc.SetDevice(1, new TV());
dc.SetDevice(2, new Radio());
dc.GetDevice(1).SwitchOn();
```

```
var dc = new DomusController(10);
dc[0] = new Lamp();
dc[1] = new TV();
dc[2] = new Radio();

dc[1].SwitchOn();
```

ESEMPIO

3-Indexers

Matrix.cs | MainIndexers.cs | Map2d.cs

3-Indexers | Indexer.Matrix<TValue> | _values

```
using System;
using System.Collections;
using System.Linq;

namespace Indexer
{
    1 reference | 0 changes | 0 authors, 0 changes
    class Matrix<TValue>
    {
        private TValue[][] _values;

        0 references | 0 changes | 0 authors, 0 changes
        public Matrix(int nRows, int nCols)
        {
            this.Cols = nCols;
            this.Rows = nRows;
            this._values = new TValue[nRows][];
            for (int i = 0; i < nRows; i++)
            {
                this._values[i] = new TValue[nCols];
            }
        }

        1 reference | 0 changes | 0 authors, 0 changes
        public int Cols { get; }
        1 reference | 0 changes | 0 authors, 0 changes
        public int Rows { get; }

        0 references | 0 changes | 0 authors, 0 changes
        public TValue this[int i, int j]
        {
            get { return this._values[i][j]; }
            set { this._values[i][j] = value; }
        }

        10 references | 0 changes | 0 authors, 0 changes
        public override string ToString()
        {
            return string.Join(
                "\n",
                this._values.Select(row => "[" + string.Join(", ", row) + "]")
```

Solution Explorer

Solution 'LezioneCSharp' (10 of 10 projects)
- Solution Items
- 0-Miscellaneous
- 1-Properties
- 2-DynamicDispatch
- 3-Indexers
  - Properties
  - References
  - App.config
  - MainIndexers.cs
  - Map2d.cs
  - Matrix.cs
- 4-OperatorsOverloading
- 5-ExtensionMethods
- 6-DelegatesAndEvents
- 7-NullablesAndOperators
- 8-Enumerables
- 9-Iterators

Overloading degli operatori

- I nuovi tipi possono ridefinire i principali operatori
  - Esempi di operatori ridefinibili: +, -, !, ~, ++, --, *, /, %, &, |, ^, <<, >>, ==, !=, <, >, <=, >=
- Gli operatori con assegnamento (es. +=) non possono essere ridefiniti direttamente
  - vengono valutati usando il corrispondente operatore binario (es. +), che può essere ridefinito.
- Per ridefinire un operatore è necessario definire un metodo «**`static`**» il cui nome è la keyword «**`operator`**» seguita dal simbolo dell'operatore.
  - Si possono anche definire metodi statici da chiamare per type-cast impliciti ed espliciti (es. «`explicit operator double` (…) » definisce un type cast esplicito a double).

```
public class Point3D
{
  private double x, y, z;

  public Point3D(double x, double y, double z)
  {
    this.x = x; this.y = y; this.z = z;
  }

  public static Point3D operator+(Point3D p1, Point3D p2)
  {
    // Esempio di operatore binario ridefinito
    return new Point3D(p1.x+p2.x, p1.y+p2.y, p1.z+p2.z);
  }

  public static Point3D operator-(Point3D p)
  {
    // Esempio di operatore unario ridefinito
    return new Point3D(-p.x, -p.y, -p.z);
  }

  public override string ToString()
  {
    return string.Format("[{0}, {1}, {2}]", x, y, z);
  }
}
```

```
var p1 = new Point3D(1, 2, 0);
var p2 = -p1; // Utilizzo operatore unario
var p3 = p1 + p2;  // Utilizzo operatore binario
p1 += p3; // Utilizzo operatore binario con assegnamento

Console.WriteLine("{0} {1}", p1, p3);
```

ESEMPIO

4-Operators Overloading

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace OperatorsOverloading
{

    class MainOperatorsOverloading
    {
        static void Main(string[] args)
        {
            StrangeInt x = 7;
            StrangeInt y = 3;

            Console.WriteLine((x + y).ToString()); // 10?
            Console.WriteLine((x - y).ToString()); // 4?
            Console.WriteLine((x * y).ToString()); // 21?
            Console.WriteLine((x / y).ToString()); // 2?

            Console.ReadKey();
            Console.WriteLine();

            int r1 = x + y;
            int r2 = x - y;
            int r3 = x * y;
            int r4 = x / y;

            Console.WriteLine(r1);
            Console.WriteLine(r2);
            Console.WriteLine(r3);
            Console.WriteLine(r4);

            Console.ReadKey();
        }
    }
}
```

Metodi di estensione

- Gli *extension methods* consentono di aggiungere metodi ai tipi esistenti, senza creare un nuovo tipo derivato
  - Es. Si vuole aggiungere un metodo personalizzato invocabile su tutti gli oggetti di tipo string senza creare un oggetto «MyString» che estenda da string.
- Parola chiave «**`this`**» sul 1° argomento
  - … di un metodo **statico** in una classe **statica**
  - Tale metodo **statico** può essere chiamato come se fosse un metodo **d'istanza** del tipo del primo parametro.

```
static class MyExtensions
{
  public static int CountOccurencies(this string text, char element)
  {
    int occurencies = 0;

    foreach (var e in text)
    {
      if (e == element
      {
        occurencies++;
      }
    }

    return occurencies;
  }
}
```

Utilizzo Classico

```
string t = "Hello, World!";

int n1 = MyExtensions.CountOccurencies(t,'l');
int n2 = MyExtensions.CountOccurencies("TESTO",'T');
```

Utilizzo dei metodi di estensione

```
string t = "Hello";

int n1 = t.CountOccurencies('l');

int n2 = "TESTO".CountOccurencies('T');
```

ESEMPIO

5-Extension Methods

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ExtensionMethods
{
    22 references | 0 changes | 0 authors, 0 changes
    sealed class Complex
    {
        17 references | 0 changes | 0 authors, 0 changes
        public double Re { get; }
        18 references | 0 changes | 0 authors, 0 changes
        public double Im { get; }

        6 references | 0 changes | 0 authors, 0 changes
        public Complex(double re, double im)
        {
            this.Re = re;
            this.Im = im;
        }

        0 references | 0 changes | 0 authors, 0 changes
        public static Complex operator +(Complex x, Complex y)
        {
            return new Complex(x.Re + y.Re, y.Im + y.Im);
        }

        0 references | 0 changes | 0 authors, 0 changes
        public static Complex operator -(Complex x, Complex y)
        {
            return new Complex(x.Re - y.Re, y.Im - y.Im);
        }

        1 reference | 0 changes | 0 authors, 0 changes
        public static Complex operator *(Complex x, Complex y)
        {
            return new Complex(x.Re * y.Re - x.Im * y.Im, x.Im * y.Re + x.Re * y.Im);
        }
```

PARTE 5

Generici, Collezioni, Delegates, Lambda, LINQ (cenni)

Genericità

Analogie

- Polimorfismo parametrico con una sintassi simile
- Possibile vincolare un parametro di tipo a essere un super-/sotto-tipo di un qualche `Type`
  - Java: `ContaierType<T extends Type>`
  - C#: `ContaierType<TItem> where TItem : Type`
- Introdotti in C# dalla versione 2.0
  - Introdotti in Java dalla versione 5
- Convenzione sul nome dei tipi generici
  - Java: singole lettere (K, V, T, R, etc)
  - C#: T seguita da nome mnemonico (TKey, TValue, TItem, ...)

Differenze

- Java utilizza «**type erasure**» durante la compilazione (i.e. dopo la compilazione si usa Object dappertutto)
  - a runtime una `List<String>` e una `List<Integer>` sono lo stesso tipo (`List<Object>`)
- In C# i tipi generici sono **reificati** a runtime
  - quindi `List<string>` e `List<object>` sono tipi diversi
- C# non supporta le «**wildcards**» nella definizioni di variabili
  - (e.g. `Set<?>`, `List<? extends X>`, `Collection<? super Y>`)
- In C# è possibile **vincolare** un parametro di tipo ad avere un costruttore pubblico senza parametri o a essere un tipo riferimento o un tipo valore.
- C# richiede che le interfacce generiche siano **definite** come covarianti, controvarianti, o invarianti rispetto a ogni parametro di tipo
  - tramite le keyword «**`in`**» (controvariante) o «**`out`**» (covariante) ...
  - ... o la loro assenza (invariante)
  - le classi generiche sono sempre invarianti

Genericità – Esempi

```java
public class Pair<X, Y>{
  private X first;
  private Y second;

  public Pair(X first, Y second){
    this.first = first;
    this.second = second;
  }

  public X getFirst() { return this.first; }

  public Y getSecond() { return this.second; }

  public String toString(){
    return "<" + first + "," + second + ">";
  }
}
```

JAVA

```java
Vector<Pair<String,Integer>> v = new Vector<>();
v.addElement(new Pair<>("Prova",1));
v.addElement(new Pair<>("di",2));
v.addElement(new Pair<>("Vettore",3));

for (Pair<String,Integer> pair : v){
  System.out.println(pair);
}
```

```csharp
public class Pair<TFirst, TSecond>
{
  private TFirst first;
  private TSecond second;

  public Pair(TFirst first, TSecond second)
  {
    this.first = first;
    this.second = second;
  }

  public TFirst GetFirst() { return this.first; }

  public TSecond GetSecond() { return this.second; }

  public override string ToString()
  {
    return "<" + first + "," + second + ">";
  }
}
```

C#

```csharp
var v = new List<Pair<string,int>>();
v.Add(new Pair<string,int>("Prova",1));
v.Add(new Pair<string,int>("di",2));
v.Add(new Pair<string,int>("Vettore",3));

foreach (var pair in v)
{
  Console.WriteLine(pair);
}
```

Collezioni e iteratori

ANALOGIE

- Ampia libreria di strutture dati e relativi algoritmi
  - namespace `System.Collections` in C# e package `java.util` in Java
- Interfacce e classi astratte per consentire di manipolare dati in modo uniforme indipendentemente dall'implementazione della specifica struttura dati
- Iterazione su qualsiasi oggetto la cui classe implementi un'apposita interfaccia
  - `Iterable<X>` in Java e `IEnumerable<X>` in C#

DIFFERENZE

- In C# alle strutture dati non generiche basate su «object» sono state affiancate classi ad hoc per tipi generici (namespace `System.Collections.Generic`)
  - mentre in Java le strutture dati non-generiche esistenti sono state adattate all'utilizzo con i tipi generici.
- In C# è possibile implementare gli iteratori in modo molto semplice scrivendo un metodo che utilizza l'istruzione «**`yield return`**»
- In C# si può creare una collezione inizializzandone gli elementi in modo simile a quanto avviene per gli array

Collezioni e iteratori – Esempio

```java
List<Integer> nums = new ArrayList<>(
        Arrays.asList(0,1,2,3,4,5,6,7,8,9));

for (int i = nums.size() - 1; i >= 0; i--){
  if (nums.get(i) % 2 == 1){
    nums.remove(i);
  }
}

Map<Integer, String> map = new HashMap<>();
for (int n : nums){
  // Associa n^3 (come testo) ad ogni numero n
  map.put(n, Integer.toString(n*n*n));
}

Random rnd = new Random();
for (int i = 0; i < 20; i++){
  int n = rnd.nextInt(10);
  String str = map.get(n);
  if (str != null){
    System.out.printf("%d ==> %s\n", n, str);
  } else {
    System.out.printf("%d non in cache.\n", n);
  }
}
```

```dotnet
IList<int> nums = new List<int> { 0,1,2,3,4,5,6,7,8,9 };

for (var i = nums.Count - 1; i >= 0; i--)
{
  if (nums[i] % 2 == 1)
  {
    nums.RemoveAt(i);
  }
}

IDictionary<int, string> map = new Dictionary<int, string>();
foreach (int n in nums)
{ // Associa n^3 (come testo) ad ogni numero n
  map.Add(n, (n*n*n).ToString());
}

var rnd = new Random();
for (int i = 0; i < 20; i++)
{
  var n = rnd.Next(10);
  string str;
  if (map.TryGetValue(n, out str))
  {
    Console.WriteLine("{0} ==> {1}", n, str);
  }
  else
  {
    Console.WriteLine("{0} non in cache.", n);
  }
}
```

Metodi anonimi (pre Lambda Expressions)

```
delegate void ProgressUpdate(int perc);

class MyExecutor
{

 public void Execute(ProgressUpdate callBack)
 {
   for (int i = 0; i < MAX; i++)
   {
      // do some stuff

     if (i % 100 == 0)
     {
       callBack(i);
     }
   }
 }

}
```

- Permettono di passare direttamente un "blocco di codice" a un parametro delegato (closure-like functions)
- Eliminano la necessità di dichiarare un metodo separato per poi poterlo passare al delegate
- La keyword **delegate** sostituisce il nome del metodo (che è appunto "anonimo") ed è seguita dalla dichiarazione degli eventuali parametri del metodo

```
static void Main(){
  MyExecutor e1 = new MyExecutor();
  e1.Execute(delegate(int p) {
                Console.Write("...{0} ", perc);
              });

  MyExecutor e2 = new MyExecutor();
  e2.Execute(delegate(int p) { });
}
```

Delegates

- Simili ai puntatori a funzione del C/C++
  - Sono Orientati agli oggetti
  - Sono Type-safe
- Analoghi alle interfacce funzionali di **Java >= 8**
  - come in Java, permettono di passare riferimenti a metodi
- Caratteristiche principali
  - Permettono di passare una funzione o un metodo come parametro o di assegnarlo a una variabile
  - Sono alla base degli eventi C#

```
class Program
{
  delegate int Operation(int a, int b);

  static int Add(int x, int y)
  {
    return x + y;
  }

  static int Sub(int x, int y)
  {
    return x - y;
  }

  static void Main()
  {
    Operation op = Add;

    int res1 = op(3, 5);

    op = Sub;

    int res2 = op(2, 4);
  }
}
```

Delegates – Altro Esempio

```
delegate void ProgressUpdate(int perc);

class MyExecutor
{

 public void Execute(ProgressUpdate callBack)
 {
   for (int i = 0; i < MAX; i++)
   {
      // do some stuff

     if (i % 100 == 0)
     {
       callBack(i);
     }
   }
 }

}
```

```
void PrintExecutorProgressStatus(int perc)
{
   Console.Write("...{0} ", perc);
}

void IgnoreProgressStatus(int perc) { }


static void Main(){
  MyExecutor e1 = new MyExecutor();
  e1.Execute(PrintExecutorProgressStatus);

  MyExecutor e2 = new MyExecutor();
  e2.Execute(IgnoreProgressStatus);
}
```

Espressioni Lambda

- Funzioni anonime che contengono istruzioni C# e possono essere utilizzate per creare delegate anonimi

- Disponibili dalla versione 3.0 del linguaggio
  - Come estensione dei delegates

- Sintatticamente molto simili alle lambda di Java >= 8
  - Operativamente, sono del tutto analoghe a Java

- Contengono l'operatore **=>**
  - La parte a sinistra dell'operatore **=>** indica i parametri della funzione
  - La parte a destra contiene un'espressione o un insieme di istruzioni racchiuso fra parentesi graffe.

```
//with parameters
n => n == 2
(a, b) => a + b
(a, b) => {a++; return a+b;}

//With explicitly typed
parameters
(int a, int b) => a + b

//No parameters
() => return 0;
```

Espressioni Lambda – Esempi

```
delegate int TipoFunzione(int parametro);
delegate int TipoFunzione2(int p1,int p2);
```

```
TipoFunzione f = x => x * x;
int a = f(5); // a = 25

TipoFunzione2 f2 = (x, y) => x + y;
int b = f2(3, 7); // b = 10
```

=

```
TipoFunzione f = delegate(int x) {
                        return x * x;
                    };

int a = f(5); // a = 25

TipoFunzione2 f2 = delegate(int x, int y) {
                            return x + y;
                        };

int b = f2(3, 7); // b = 10
```

```
e.Execute(p => Console.Write("...{0} ", p));
```

=

```
e.Execute(delegate(int p) {
                Console.Write("...{0} ", p);
            });
```

Events

- Gli eventi permettono a un oggetto di notificare altri quando avviene una situazione di interesse
  - La classe che emette l'evento è detta *publisher*, mentre le istanze della classe che lo gestiscono sono dette *subscriber*
  - Molto usati in contesti di GUI, sia desktop (Windows Forms, WPF) che web (ASP.NET)
- Il publisher determina quando l'evento viene emesso, indipendentemente dai subscriber
- A un evento si possono registrare molteplici subscriber
  - Il publisher non conosce i subscriber: è sufficiente che questi registrino un proprio delegate perché vengano notificati
  - Un subscriber può registrare azioni per diversi eventi su diversi publisher
  - Quando più subscriber sono registrati per il medesimo evento, gli event handler sono generalmente invocati in modo sincrono
- Un evento che non ha subscribers non viene emesso
- Per semplificare l'implementazione, la libreria fornita con .NET espone:
  - il delegate EventHandler e la versione con tipo generico EventHandler<TEventArgs>
  - la classe EventArgs per contenere i dati dell'evento

Events – Esempio

```
public class EventEmitter
{
    public event EventHandler OnSomethingHappened;

    public event EventHandler<string> OnSomethingElseHappened;

    // [...]

    private void DoThings()
    {
        // [...] do something

        // if there are subscribers, the event is not null
        if (OnSomethingHappened != null)
        {
            this.OnSomethingHappened(this, EventArgs.Empty);
        }

        // you can also use null propagation check:
        this.OnSomethingElseHappened?.Invoke(this, "foo");
    }
}
```

```
public class EventSubscriber
{
    public EventSubscriber(EventEmitter publisher)
    {
        // you can register a method delegate ...
        publisher.OnSomethingHappened += this.HandleEvent;

        // ... or an anonymous delegate ...
        publisher.OnSomethingElseHappened +=
            delegate(object sender, string args) { Console.Write(args); };

        // ... o a lambda delegate ...
        publisher.OnSomethingHappened += (sender, args) => Console.Write(args);
    }

    public void HandleEvent(object emitter, EventArgs args)
    {
        Console.WriteLine("Event received");
    }
}
```

Nullable types

- Una variabile di tipo `Nullable<T>` può assumere tutti i valori di T oppure il valore `null` (es. `Nullable<bool>` può assumere tre valori: `true`, `false`, `null`).
  - T deve essere un value type
- La sintassi `T?` (es. `bool?`, `int?`, …) equivale a `Nullable<T>` ed è preferibile perché più intuitiva e leggibile.
- Utili in tutti i casi in cui un value type può avere valore non-definito (caso tipico: quando si interagisce con un database).
- Metodi `GetValueOrDefault()`, proprietà `HasValue(bool)` e `Value(T)`.
- L'operatore **??** (null-coalescing) consente di specificare un valore di default da restituire nel caso in cui la variabile sia null.

```
int? v1 = null; // equivale a Nullable<int> v1 = null;
int? v2 = 42;   // equivale a Nullable<int> v2 = 42;

Console.WriteLine("V1: {0}", v1!=null ? v1.ToString() : "Sconosciuto"); // oppure ..., v1.HasValue ?
Console.WriteLine("V2: {0}", v2!=null ? v2.ToString() : "Sconosciuto");
Console.WriteLine(v1.GetValueOrDefault()); // 0 (default di int)
Console.WriteLine(v2.GetValueOrDefault()); // 42
Console.WriteLine(v1 ?? 3);  // 3
Console.WriteLine(v2 ?? 3);  // 42
Console.WriteLine((v1 + 3) == null);  // True (null + 3 = null)
Console.WriteLine((v2 + 58) == null);  // False (42 + 58 = 100)
```

C# `IEnumerable` VS Java8 `Stream`

`IEnumerable` ≈ `Iterable` + `Stream`

C# Java 8

Classe «`System.Linq.Enumerable`»

- Fornisce molti *extension methods* che implementano molte funzioni *high-order* per manipolare flussi di dati (potenzialmente illimitati)
- Permette di costruire *pipeline* di elaborazione analogamente a `Collection.stream()` di Java 8
- Anche gli array implementano `IEnumerable`!
- Nomenclatura più vicina al mondo SQL che al mondo funzionale
  - `Enumerable.Select` ≈ `Stream.map`
  - `Enumerable.Aggregate` ≈ `Stream.reduce | Stream.collect`
  - `Enumerable.Where` ≈ `Stream.filter`
  - …

Language-Integrated Query – LINQ (1/2)

- Integrazione all'interno del C# di un linguaggio di interrogazione simile a SQL
  - Introdotto per semplificare la gestione delle collezioni di dati
- Basato su:
  - Variabili implicitamente tipizzate e inizializzatori di oggetti
  - Tipi generici, metodi di estensione e tipi anonimi
  - Espressioni lambda
  - Apposite keyword e sintassi per scrivere le query
- Cosa si può interrogare?
  - Database
  - Strutture dati in memoria, collections, array, …
  - Documenti XML
  - Ogni altra fonte dati che implementi un «provider LINQ»

```
//  1. Fonte dati: array in memoria
int[] numbers = new int[7] {0,1,2,3,4,5,6};

// 2. Creazione della query
var numQuery = from num in numbers
               where (num % 2) == 0
               orderby num descending
               select num;

// 3. Esecuzione della query
foreach (int num in numQuery)
{
  Console.WriteLine(num);
}
```

```
// 1. Fonte dati: file xml
var xml = XElement.Load(@"C:\Temp\CD.xml");

// 2. Creazione della query
var xmlQuery = from cd in xml.Elements()
               where cd.Element("COUNTRY").Value == "UK"
               group cd by cd.Element("YEAR").Value into g
               orderby g.Key
               select new
               {
                 Anno = g.Key,
                 Numero = g.Count(),
               };

// 3. Esecuzione della query
foreach (var r in xmlQuery)
{
  Console.WriteLine("{0}: {2} CD", r.Anno, r.Numero);
}
```

Language-Integrated Query – LINQ (2/2)

Query operators

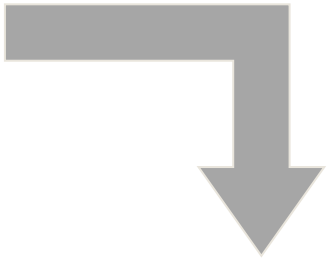
- Metodi che forniscono le funzionalità del LINQ (es. selezione, raggruppamento, …)
- Implementati come metodi di estensione generici che agiscono su interfacce (`IEnumerable<T>` e `IQueryable<T>`)
- La sintassi «`from` … `where` … `select`» viene trasformata dal compilatore in chiamate a tali metodi
  - il programmatore può anche utilizzarli direttamente
- N.B. L'esecuzione delle operazioni che restituiscono una sequenza di elementi non avviene al momento della chiamata del metodo, ma è posticipata al momento in cui si enumera il risultato.

```
// sintassi «query expression»
var numQuery = from num in numbers
               where (num % 2) == 0
               orderby num descending
               select num;
```

```
// sintassi «method-based»
var numQuery = numbers
      .Where(num => (num % 2) == 0)
      .OrderByDescending(num => num)
      .Select(num => num);
```

LINQ – Altro Esempio

```
var nums = new List<int> {0,1,2,3,4,5,6,7,8,9};

for (var i = nums.Count - 1; i >= 0; i--)
{
  if (nums[i] % 2 == 1)
  {
    nums.RemoveAt(i);
  }
}

var map = new Dictionary<int, string>();

foreach (var n in nums)
{ // Associa n^3 (come testo) ad ogni numero n
  map.Add(n, (n*n*n).ToString());
}
```



```
var nums = Enumerable.Range(0, 10);
var map = nums.Where(n => n % 2 == 0)
        .ToDictionary(n => n, n => (n * n * n).ToString());
```

Custom stream: **Iteratori** + metodi estensione

C# supporta gli iteratori a livello di linguaggio tramite la sintassi «**`yield return`**»

- Applicabile in metodi aventi come tipo di ritorno `IEnumerable<T>` o `IEnumerator<T>`
  - Si usa «**`yield return`**» al posto di «**`return`**»
- Lo stato del motodo «sopravvive» alla «**`yield return`**»
  - Il compilatore genera un o `IEnumerator<T>` dietro le quinte
- Usato in metodi estensione, permette di estendere a piacere l'insieme di operazioni high-order possibili sugli enumerabili
  - In Java, sarebbe come poter aggiungere metodi all'interfaccia `Stream`
- LINQ è implementato combinando iteratori e metodi estensione
  - Si veda la classe «`System.Linq.Enumerable`»

Esempio: custom map & filter in C#

```
public static IEnumerable<TOut> Map<TIn, TOut>(this IEnumerable<TIn> sequence, Func<TIn, TOut> mapping)
{
    foreach (var element in sequence)
    {
        yield return mapping(element);
    }
}
```

```
public static IEnumerable<TAny> Filter<TAny>(this IEnumerable<TAny> sequence, Predicate<TAny> predicate)
{
    foreach (var element in sequence)
    {
        if (predicate(element))
        {
            yield return element;
        }
    }
}
```

PARTE 6

Thread e Concorrenza

Thread e concorrenza

ANALOGIE

- Supporto a livello di linguaggio con keyword per definire blocchi di codice come sezioni critiche
  - «`synchronized`» in Java, «**`lock`**» in C#
  - keyword «`volatile`» per proibire ottimizzazioni del compilatore che comporterebbero problemi in caso di accessi concorrenti.
- Possibilità di definire interi metodi come regioni critiche
  - keyword «`synchronized`» in Java, attributo `[MethodImpl(MethodImplOptions.Synchronized)]` in C#
- Ricche librerie di classi con costrutti e strutture dati per semplificare lo sviluppo di applicazioni concorrenti
  - es. **`java.util.concurrent`** in Java, **`System.Threading`** e **`System.Collections.Concurrent`** in C#

DIFFERENZE

- I thread di Java accettano `Runnables`, i thread di .NET accettano **`delegates`**
- In Java ogni classe eredita `wait()`, `notify()` and `notifyAll()` da Object
- In C# i metodi equivalenti sono `Wait()`, `Pulse()` and `PulseAll()` della classe «`System.Threading.Monitor`»

Thread e concorrenza – Esempio

```java
void action() {
    System.out.println("print on different thread");
}

// [...]

Runnable action = this::action;

// or anonimously as ...

action = () -> System.out.println("print on different thread");

new Thread(action).start();
```

```dotnet
using System;
using System.Threading;

// [...]

public void Action()
{
    Console.Write("print on different thread");
}

// [...]

// ThreadStart is a delegate, equivalent to Runnable
ThreadStart action = this.Action;
                     // ^ we store the run method as a delegate;
                     //   note that parenthesis are missing,
                     //   like in Java with method references

// or anonimously as delegate ...

action = delegate { Console.Write("print on different thread"); };

// or anonimously as lambda ...

action = () => Console.Write("print on different thread");

// in both cases ...

new Thread(action).Start();
```

PARTE 7

Test

Unit Testing

- Come per Java, anche in C# sono disponibili diverse librerie per scrivere test automatici
  - Le tre librerie principali hanno sintassi diverse fra loro, ma a livello di funzionalità sono quasi equivalenti
  - Inoltre, sono parte della .NET Foundation

- **MSTest v2** (https://github.com/Microsoft/testfx) è un progetto open-source sviluppato e mantenuto da Microsoft;
  - è la soluzione "ufficiale", in quanto evoluzione del tradizionale **Visual Studio Unit Testing Framework** integrato nell'IDE

- **Nunit** (https://nunit.org) è un progetto open-source sviluppato e mantenuto da community;
  - nato come porting di JUnit per .NET, dalla versione 3 è stato completamente riscritto per offrire un miglior supporto a tutte le piattaforme .NET
  - attualmente è la meno usata delle tre

- **xUnit** (https://xunit.net) è un progetto open-source nato dall'inventore di NUnit 2 e mantenuto da community;
  - è probabilmente la più diffusa, tanto che persino Microsoft la utilizza in alcuni suoi progetti open-source
  - presenta una sintassi leggermente diversa dalle altre

- In questa lezione sarà trattato **MSTest**, in quanto offre una sintassi molto simile a JUnit ed è tutt'ora rilevante

Unit Testing - Esempio

```java
import static org.junit.Assert.*;
import org.junit.Test;

public class SampleTest {
  @Test
  public void fail() {
    fail("We want to fail");
  }

  @Test
  public void DoubleEquals() {
    assertEquals(0.3, 0.1 + 0.2, 0.00001);
  }

  @Test
  public void ConditionCheck() {
    var list = List.of(1, 2);
    assertTrue(!list.isEmpty());
    assertFalse(list.stream().anyMatch(num -> num > 2));
  }
}
```

```csharp
using Microsoft.VisualStudio.TestTools.UnitTesting;


[TestClass]
public class SampleTest
{
  [TestMethod]
  public void Fail()
  {
    Assert.Fail("We want to fail");
  }

  [TestMethod]
  public void DoubleEquals()
  {
    Assert.AreEqual(0.3, 0.1 + 0.2, 0.00001);
  }

  [TestMethod]
  public void ConditionCheck()
  {
    var list = new List<int> { 1, 2 };
    Assert.IsTrue(list.Any());
    Assert.IsFalse(list.Any(num => num > 2));
  }
}
```

Riferimenti

MSDN - Visual C#

- https://docs.microsoft.com/en-us/dotnet/csharp/
- https://msdn.microsoft.com/en-us/library/67ef8sbd.aspx
- https://msdn.microsoft.com/en-us/library/618ayhy6.aspx